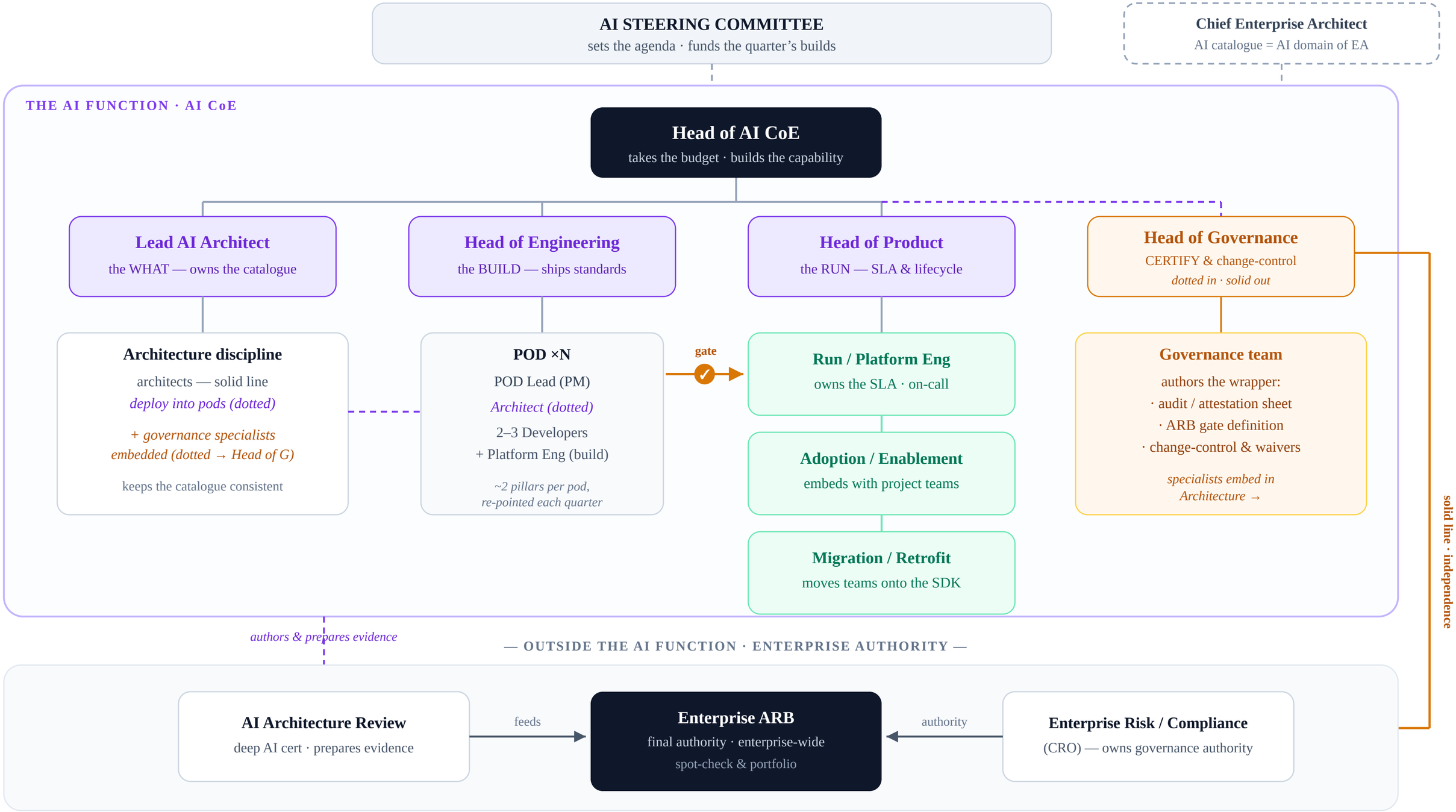
PART 5

CLOSING UP


AI STEERING COMMITTEE
sets the agenda · funds the quarter's builds
Chief Enterprise Architect
AI catalogue = AI domain of EA
THE AI FUNCTION · AI CoE
Head of AI CoE
takes the budget · builds the capability
Lead AI Architect
the WHAT — owns the catalogue
Head of Engineering
the BUILD — ships standards
Head of Product
the RUN — SLA & lifecycle
Head of Governance
CERTIFY & change-control
dotted in · solid out
Architecture discipline
architects — solid line
deploy into pods (dotted)
+ governance specialists
embedded (dotted → Head of G)
keeps the catalogue consistent
POD ×N
POD Lead (PM)
Architect (dotted)
2–3 Developers
+ Platform Eng (build)
~2 pillars per pod,
re-pointed each quarter
gate
Run / Platform Eng
owns the SLA · on-call
Adoption / Enablement
embeds with project teams
Migration / Retrofit
moves teams onto the SDK
Governance team
authors the wrapper:
· audit / attestation sheet
· ARB gate definition
· change-control & waivers
specialists embed in
Architecture →
solid line · independence
authors & prepares evidence
— OUTSIDE THE AI FUNCTION · ENTERPRISE AUTHORITY —
AI Architecture Review
deep AI cert · prepares evidence
feeds
Enterprise ARB
final authority · enterprise-wide
spot-check & portfolio
authority
Enterprise Risk / Compliance
(CRO) — owns governance authority

The Four Owners — Roles & Responsibilities

The companion to the org chart — what each head owns, and the independence that keeps the verdict honest.

DEFINE · THE WHAT

the design authority

Lead AI Architect

Reports to Head of AI CoE · dotted to Chief EA

OWNS

- Reference architectures — the blessed AI patterns.
- The standards catalogue — holds the pen on content.
- Tech & model selection — recommend and ratify.

DOESN'T OWN

build, run, or governance sign-off.

ACCOUNTABLE FOR

Technical correctness & a coherent estate.

BUILD · THE MACHINE

the delivery muscle

Head of Engineering

Reports to Head of AI CoE

OWNS

- The AI platform — gateway, SDK, MLOps.
- Pods that build standards into enforceable RIs.
- Eng quality, CI/CD, the delivery schedule.

DOESN'T OWN

the standard, the SLA, or certifying.

ACCOUNTABLE FOR

Shipping operable builds — the readiness gate.

OPERATE · THE RUN

platform as a product

Head of Product

Reports to Head of AI CoE

OWNS

- Run & SLA — uptime, lifecycle, support.
- Adoption + brownfield SDK migration.
- FinOps / chargeback — cheaper than DIY.

DOESN'T OWN

standard content, SDK build, waivers.

ACCOUNTABLE FOR

Adoption & reliability — enable before enforce.

JUDGE · CERTIFY

independent — solid line to CRO

Head of Governance

Reports to CRO — outside AI · dotted to CoE

OWNS

- Responsible-AI policy & regulatory mapping.
- Certification, change-control & the audit trail.
- Model risk, waivers, the risk register.

DOESN'T OWN

standard content or final sign-off (ARB).

ACCOUNTABLE FOR

Certification credible to a regulator.

Four owners, on purpose — define · build · operate · judge.

Architecture DEFINES the bar | **Engineering BUILDS the machine** | **Product RUNS it as a service** | **Governance OWNS the verdict**

The AI function authors and prepares; the Enterprise ARB approves — they never merge.

The moment the team that writes the standard also signs it off, the independence that made governance worth anything is gone.

The Enforcement Journey — Attest → Verify → Provide

Each rung is stronger, costlier, and harder to commit to. The control only gets real as you climb — you earn each rung, one at a time.

STRONGER · COSTLIER · HARDER TO COMMIT ↗

LEVEL 1 · ATTEST

ENFORCEMENT STRENGTH

EFFORT: LOW

WHAT IT IS

The project self-certifies — a guardrail YAML, every threat dispositioned (mitigated / accepted / N-A), signed at ARB.

★ GATE

Completeness check: every threat dispositioned + signed.
Not efficacy.

Proves it was claimed — not that it works.

⚠️ **Most enterprises stop here.**

LEVEL 2 · VERIFY

ENFORCEMENT STRENGTH

EFFORT: HIGH

WHAT IT IS

EA fires an adversarial corpus at the running endpoint and scores it. Only probe-testable controls qualify.

★ GATE

0 successful overrides on the central corpus + false-positive rate under threshold.

Claim replaced by proof — it actually works.

Needs a central platform.

LEVEL 3 · PROVIDE

ENFORCEMENT STRENGTH

EFFORT: HIGHEST

WHAT IT IS

A central SDK / gateway does the filtering, scanning, tool-auth, logging. SDK alone isn't L3 — only SDK + enforced routing.

★ GATE

Egress / IAM blocks every direct model call — routing proof + provenance + signed attestation.

Non-bypassable — compliance = "routed?"

Needs platform + egress control.

The catalogue isn't the hard part. This climb is.

Architecture defines the bar | **Engineering builds the machine** | **Product runs it** | **Governance owns the verdict**

Enable before you enforce — enterprise AI fails when no one commits past Level 1.

Each rung is a fresh commitment: the utility built, and the estate supported onto it, before the gate turns mandatory.

Level 1 · Attest — Self-Certification

Rung 1 of 3 · the project certifies itself against the standard — cheap, immediate, no tooling. It proves the claim was made, not that the control works.

LEVEL 1
ATTEST

RUNG 1 OF 3

ENFORCEMENT STRENGTH

Self-certification

The project certifies itself against the standard.

EFFORT: LOW · enforceable today, no tooling

WHAT IT IS

A guardrail YAML per workload — every threat dispositioned (mitigated / accepted / N-A), a named owner, and the whole file signed at the ARB.

Coverage first — every relevant standard gets a decision on the record.

★ THE GATE

Every threat dispositioned + signed — a completeness check, run manually or by a ~20-line CI linter.

It checks the form is complete — not that the control works.

Claim, not efficacy. That is the ceiling of this rung.

guardrail.yaml *one per workload*

```
workload: claims-triage-agent
owner: jane.doe@bank.com
standards:
  GO1B1-01: # eval harness
    disposition: mitigated
  GS1B3-02: # prompt-injection
    disposition: accepted
    rationale: low blast radius
  GC2B2-01: # cost ceiling
    disposition: na
arb_signoff: 2026-06-14
```

WHO DOES WHAT — AT LEVEL 1

Architecture
DEFINE

Owns the threat taxonomy & standards; sets the applicability metadata that filters the list to what actually applies.

Governance
JUDGE

Runs the ARB review; challenges N-A & accepted dispositions; certifies; wires to post-incident.

Independent.

Engineering
BUILD

Minimal at this rung — optionally a ~20-line completeness linter for the YAML.

No platform needed.

Product
OPERATE

Owns the attestation template + submission process; tracks which workloads are signed.

Project
SURFACE + SIGN

Authors the YAML, dispositions every relevant standard, names the owner, and signs.

What Attest buys you — and its ceiling

BUYS ✓

Coverage across the estate immediately · a named owner per workload · an audit trail the ARB can sign · the habit of dispositioning every relevant standard.

CEILING ✗

It proves the claim, not the control — a signed YAML can still describe a system that doesn't work.

⚠️ Most enterprises stop here.

Commitment ends at the cheapest rung — and that is why AI fails beyond the demo.

The floor, not the finish. Next: Verify →

Level 2 · Verify — Trust, but Verify

Rung 2 of 3 · EA attacks the running system, not the paperwork — the claim is replaced by proof. Costs a central platform.

LEVEL 2
VERIFY

RUNG 2 OF 3

ENFORCEMENT STRENGTH

Adversarial test

EA attacks the running system, not the paperwork.

EFFORT: HIGH · needs a central platform

WHAT IT IS

EA fires an adversarial corpus at the deployed endpoint and scores the responses. The project can't fake it — only probe-testable controls qualify.

The claim is replaced by proof that the control actually holds.

★ THE GATE

Zero successful overrides on the central corpus, and a false-positive rate under threshold.

Efficacy, not paperwork — it has to actually hold.

Point-in-time proof: true at the moment of the run.

verify-run · adversarial corpus *point-in-time*

target: claims-triage-agent (staging)
suite: prompt-injection v14
corpus: 480 adversarial prompts

successful_overrides: **0** # gate: must be 0
false_positive_rate: **1.8%** # < 3%
probe_testable: **true**

verdict: **PASS** → certified, release unblocked

One override in the corpus = FAIL. Governance owns this verdict, not the project.

WHO DOES WHAT — AT LEVEL 2

Architecture
DEFINE

Defines "pass": the threshold (0), what counts as an override, and which controls are probe-testable.

Governance
JUDGE

Owns the verdict + gate. Consumes the report, certifies, blocks release. *Independent.*

Engineering
BUILD

Builds the harness, corpus library + generator, and the scoring / oracle engine; keeps the corpus current.

Product
OPERATE

Runs it as a service: schedules runs, SLA, cost pool, reporting; tracks verification coverage.

Project
SURFACE + FIX

Staging endpoint + creds, the invariants / oracle, a ~15-line adapter; fixes what fails.

What Verify buys you — and what it costs

BUYS ✓

Proof the control actually holds under attack · failures caught before release · an independent, evidence-backed verdict the ARB can trust — not a self-signed claim.

CEILING ✗

Point-in-time, not runtime — nothing stops a bypass between runs. Only probe-testable controls qualify.

⚙ The costly leap.

Verify needs a built platform + a living corpus — the investment most orgs stall before.

The runtime guarantee is next: Provide →

Level 2 · Verify — How the Harness Actually Runs

One central tool tests every agent through a fixed contract. Central builds the machine; local surfaces the agent and its own rules.

CENTRAL PLATFORM · eval-core — built once, used by every project

Engineering builds it · Architecture defines the gate · Product runs it as a service

Corpus Library (+ generator)

Shared adversarial sets — versioned; grown from every incident.

Harness / Runner

For each scenario: build a standard request, call the endpoint, collect the response.

① request (schema) →

← ② response (schema)

Scoring Engine

Grades each response — was there a successful override?

- canary · schema
- PII classifier · LLM-judge

■ deterministic first

■ probabilistic only when needed

Consumes the project's domain invariants to judge business-rule outcomes it can't know generically.

★ Gate & Thresholds

overrides == 0

FP-rate < 3%

Report / Evidence

dashboard · CI status · ARB evidence

domain oracle

THE CONTRACT ≡ THE STANDARD

fixed request / response schema

LOCAL PROJECT — wire in, don't rebuild

surfaces its agent + its own business rules

Adapter

~15 lines

Maps the standard request ↔ your agent's real API (auth, session, invocation).

Agent under test

Staging endpoint + creds — the live deployed system.

Domain invariants + project cases

Business truths only you know — claims > £10k never auto-approve

— plus your own attack cases, authored in the shared schema.

THE VERDICT

PASS · 0 overrides → release unblocked, evidence filed

FAIL · ≥1 override → release blocked

Governance owns this verdict — not the project. It consumes the report; it doesn't run the harness.

Fix the contract once —

and one tool verifies the whole estate.

Each team adds only a thin adapter and its own rules — nothing else.

Level 3 · Provide — Non-Bypassable Infrastructure

Rung 3 of 3 · EA owns the control as infrastructure — compliance = “routed?”. The summit, and the costliest to reach.

LEVEL 3
PROVIDE

RUNG 3 OF 3

ENFORCEMENT STRENGTH

Non-bypassable infra

The control becomes infrastructure, not a check.

EFFORT: HIGHEST · platform + egress control

WHAT IT IS

A central SDK / gateway does the filtering, scanning, tool-auth, and logging. SDK alone isn't L3 — only SDK + enforced egress routing.

The control becomes infrastructure — impossible to skip.

★ THE GATE

Egress / IAM policy blocks every direct call to the model — routing proof + provenance + attestation.

Compliance = “routed?” — a runtime guarantee.

Continuous, not point-in-time. A bypass is impossible.

routing-proof · egress policy *continuous*

```
gateway: ai-gateway.internal
egress: deny-all model endpoints
allow: [ ai-gateway.internal ] # only exit

direct_model_calls: 0 # blocked at IAM
calls_via_gateway: 100% # routed
provenance: signed # sigstore

verdict: PROVIDED → non-bypassable
```

There is no un-routed path out. The policy, not the project, guarantees it.

WHO DOES WHAT — AT LEVEL 3

Architecture
DEFINE

Owns what the SDK / gateway must enforce, and the routing policy's required behaviour.

Governance
JUDGE

Gate = routing proof + provenance + signed attestation. Certifies; owns waivers for genuine gaps.

Engineering
BUILD

Builds + maintains the SDK, gateway, egress / IAM enforcement, filtering, and migration tooling.

Product
OPERATE

Operates the gateway: SLA, adoption + brownfield migration, FinOps; enable before enforce.

Project
ROUTE + MIGRATE

Routes all model calls through the sanctioned client; keeps domain controls; migrates legacy calls.

What Provide buys you — and what it takes

BUYS ✓

The control is non-bypassable — compliance = “routed?” · a continuous runtime guarantee, not a point-in-time check · the compliant path becomes the default path.

TAKES ▲

Highest cost — platform + egress control + the brownfield migration of every legacy call.

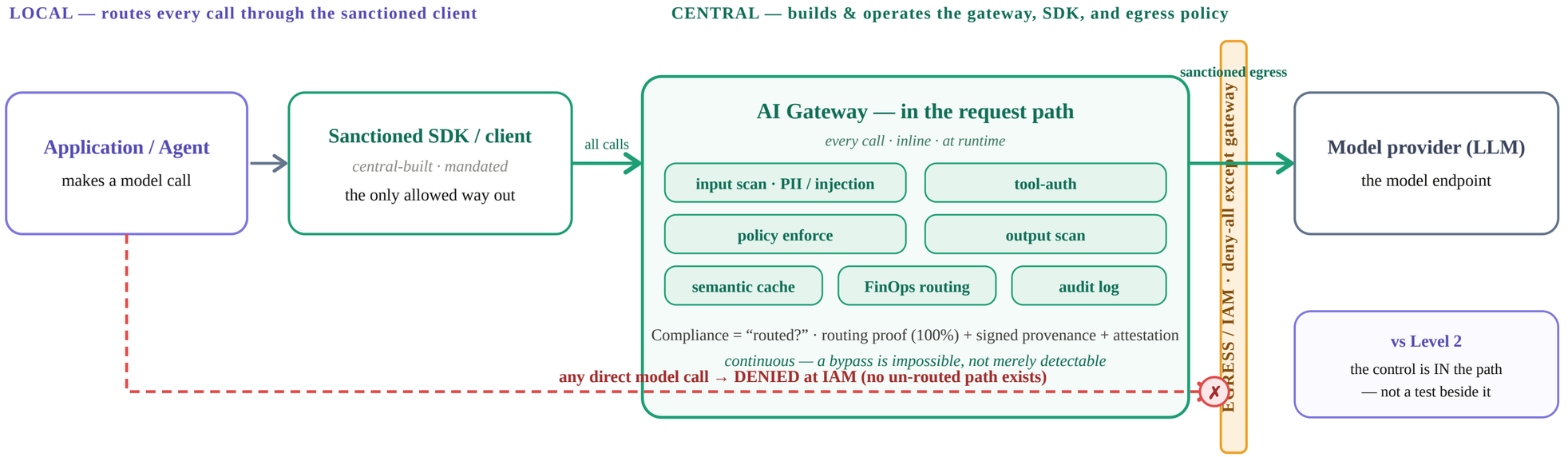
★ The summit — enforced by infrastructure.

Not goodwill. Reachable only after the SDK is supportable and every legacy call is migrated.

Enable, then enforce — that is the whole climb.

Level 3 · Provide — In-Band Gateway (the control is now in the request path)

Every model call is routed through a central gateway. A direct call is impossible — egress is blocked at the network. Compliance = “routed?”.



Level 2 → Level 3 — what actually changes

	LEVEL 2 · VERIFY — out-of-band test	LEVEL 3 · PROVIDE — in-band gateway
When	Test time — scheduled or pre-release	Run time — on every single call
In the request path?	No — a harness beside the app	Yes — a gateway the call passes through
Coverage	A sampled adversarial corpus	100% of live production traffic
Bypassable?	Yes — a direct call between runs slips by	No — egress / IAM blocks any direct call
Proves	It worked at the moment of the run	It can't not work — enforced by infra
Control lives in	A test harness	The data path (gateway + egress)